

# **Trabajo Sistemas Operativos 2: parking.c práctica Linux**

Sistemas Operativos II GIISI  
USAL

Grupo: G08

Alumno 1: Brais Bértolo Senra

Alumno 2: Juan Riego Vila

# Índice

|                                                               |   |
|---------------------------------------------------------------|---|
| 1. RECURSOS IPC USADOS, VALORES INICIALES Y SIGNIFICADO ..... | 3 |
| 1. 1. Buzón de Mensajes (id_buzon) .....                      | 3 |
| 1. 2. Memoria Compartida (id_mem) .....                       | 3 |
| 1. 3. Semáforos (id_sem) .....                                | 3 |
| 1. 3. 1. PSEUDOCÓDIGO PROCESO GESTOR Y PROCESO CHOFER .....   | 3 |
| 1. 4. Función crear_chofer .....                              | 4 |
| 1. 4. 1. PSEUDOCODIGO FUNCIÓN CREAR_CHOFER .....              | 5 |
| 1. 5. Función proceso_gestor() .....                          | 5 |
| 1. 5. 1. PSEUDOCODIGO FUNCIÓN PROCESO_GESTOR .....            | 6 |
| 1. 6. Función proceso_avisador() .....                        | 6 |
| 1. 6. 1. PSEUDOCODIGO FUNCIÓN PROCESO_AVISADOR .....          | 7 |

# 1. RECURSOS IPC USADOS, VALORES INICIALES Y SIGNIFICADO

## 1. 1. Buzón de Mensajes (**id\_buzon**)

- **Uso:** Comunicación entre la Biblioteca, el Gestor y los Chóferes.
- **Mensajes tipo 100:** Peticiones originales enviadas por la biblioteca.
- **Mensajes tipo 1 (Alta) y tipo 2 (Baja):** Peticiones reenumeradas por el Gestor según la política (PA, PD o FIFO) para que el SO las ordene automáticamente.

## 1. 2. Memoria Compartida (**id\_mem**)

- **acera[4][80] y carril[4][80]:** Matrices para guardar la ocupación física del parking por cada algoritmo.
- **proxAparcar[4]:** Vector para guardar el puntero circular necesario en el algoritmo de Siguiente Ajuste.

## 1. 3. Semáforos (**id\_sem**)

- **SEM\_MUTEX (Valor inicial = 1):** Garantiza la exclusión mutua estricta al leer o escribir en la Memoria Compartida (acera, carril, proxAparcar).
- **SEM\_ORDEN[4] (Valor inicial = 1 para cada uno):** Array de 4 semáforos. Garantiza que no haya dos chóferes calculando huecos para el mismo algoritmo a la vez, evitando que se asigne el mismo sitio a dos coches distintos.
- **SEM\_AVANCE[4] (Valor inicial = 0):** Array de 4 semáforos usado en las funciones de permiso\_avance para sincronizar los pasos de la simulación gráfica de la biblioteca.

### 1. 3. 1. PSEUDOCÓDIGO PROCESO GESTOR Y PROCESO CHOFER

**Nota:** Se asume que la Biblioteca internamente hace un `Send(Buzon, msg, tipo=100)` cuando llega un coche nuevo o agota su tiempo.

| PROCESO GESTOR<br>=====                                                                                                                                                                                                                                                                                                                                  | PROCESO CHÓFER<br>=====                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> Por_siempre_jamás {     // Lee SÓLO peticiones de la biblioteca     los     Receive(Buzon, msg, tipo=100)      // Renumerar según política (PA/PD/FIFO)     Si (cumple_prioridad)         msg.tipo = 1 // Alta prioridad     Sino         msg.tipo = 2 // Baja prioridad      // Reenvía a la cola para los chóferes     Send(Buzon, msg) } </pre> | <pre> Por_siempre_jamás {     // Pide tipo -2: El SO le da primero     // de tipo 1 y luego los de tipo 2     Receive(Buzon, msg, tipo=-2)      alg = obtener_algoritmo(msg.coche)      // Evita que dos chóferes del mismo     // algoritmo actúen a la vez     Wait(SEM_ORDEN[alg])      // Protege la memoria compartida     Wait(SEM_MUTEX)      Si (msg.subtipo == APARCAR) {         pos = buscar_hueco(alg)         escribir_acera_shm(pos)     } Sino {         borrar_acera_shm(pos)     }      // Libera la memoria compartida     Signal(SEM_MUTEX)      // Llama a la biblioteca para animar     Ejecutar_Operacion_Biblioteca()      // Libera el turno de su algoritmo     Signal(SEM_ORDEN[alg]) } </pre> |

## 1. 4. Función crear\_chofer

Es la función que crea los procesos trabajadores para que los coches realicen la función de aparcar y desapparcar.

### Funcionamiento interno:

- **Bifurcación:** Ejecuta la llamada al sistema `fork()`.
- **Configuración del Hijo:** El proceso hijo resultante hereda el acceso a los semáforos, el buzón y la memoria compartida. Inmediatamente invoca a `bucle_chofer()`, donde entrará en su ciclo de trabajo.
- **Gestión del Padre:** El proceso padre almacena el PID del nuevo chófer en un array global. Esto es fundamental para poder realizar un `wait()` o enviar señales de terminación (`SIGTERM`) a todos los hijos cuando la simulación deba cerrarse definitivamente.
- **Importancia:** Abstrae la creación de procesos, permitiendo que el programa principal lance tantos chóferes como se haya especificado por línea de comandos de forma limpia y organizada.

### 1. 4. 1. PSEUDOCODIGO FUNCIÓN CREAR\_CHOFER

```
PROCESO CREACIÓN CHOFER
=====
Por_siempre_jamás
{
    // El S0 extrae automáticamente primero los de tipo 1 y luego los de tipo 2
    Receive(Buzon, msg, tipo_solicitado=-2)

    alg = obtener_algoritmo(msg.coche)

    // 1. Bloqueo de Algoritmo: Evita que dos chóferes del mismo algoritmo colisionen
    Wait(SEM_ORDEN[alg])

    // 2. Bloqueo de Memoria: Exclusión mutua para la memoria compartida
    Wait(SEM_Mutex)

    Si (msg.subtipo == APARCAR) {
        pos = buscar_hueco(alg)
        escribir_acera_shm(pos)
    } Sino {
        borrar_acera_shm(pos)
    }

    // Libera la memoria compartida
    Signal(SEM_Mutex)

    // Llama a la biblioteca (la simulación visual no necesita mutex de memoria)
    Ejecutar_Operacion_Biblioteca()

    // Libera el turno para el siguiente chófer de su mismo algoritmo
    Signal(SEM_ORDEN[alg])
}
```

### 1. 5. Función proceso\_gestor()

Esta función actúa como el cerebro logístico de la cola de mensajes. Su objetivo es transformar una cola FIFO estándar en una cola de prioridad dinámica.

#### Funcionamiento interno:

- **Intercepción:** Realiza un bloqueo en msgrcv esperando únicamente mensajes de tipo PARKING\_MSG (tipo 100), que son los que genera la biblioteca original.
- **Lógica de Clasificación:** Mediante una estructura condicional, comprueba si la petición es de «Aparcar» o «Desaparcar». Cruza este dato con las banderas de política de la práctica (prio\_PA o prio\_PD).
- **Re-etiquetado:** Si el mensaje debe tener prioridad, cambia su campo msg.tipo a 1 (Alta). Si es la operación secundaria, le asigna el tipo 2 (Baja).
- **Inyección:** Devuelve el mensaje modificado al buzón con msgsnd.
- **Importancia:** Permite que los chóferes no tengan que «pensar» en las prioridades; simplemente piden el mensaje con el número de tipo más bajo al Sistema Operativo.

### 1. 5. 1. PSEUDOCODIGO FUNCIÓN PROCESO\_GESTOR

```
Por_siempre_jamás
{
    // Espera bloqueado hasta recibir una petición original de la biblioteca
    Receive(Buzon, msg, tipo_solicitado=100)

    Si (la operacion coincide con la política de prioridad elegida)
        msg.tipo = 1 // Asigna Alta Prioridad
    Sino
        msg.tipo = 2 // Asigna Baja Prioridad

    // Reenvía el mensaje a la cola para que lo coja un chófer
    Send(Buzon, msg)
}
```

### 1. 6. Función proceso\_avisador()

Encargada de la gestión del tiempo de la simulación.

#### Funcionamiento interno:

- **Temporización:** Utiliza `alarm(30)` para activar una señal `SIGALRM` en el sistema operativo pasados 30 segundos de ejecución, y luego `pause()` para mantener (no se si mantener es la mejor palabra, por si se te ocurre alguna que creas que es mejor) el proceso sin consumir CPU hasta que se active la señal.
- **Finalización Controlada:** Una vez agotado el tiempo, este proceso realiza la llamada a `PARKING_fin(1)`. Esto le indica a la biblioteca que no acepte más coches nuevos, pero permite que los que están dentro terminen de salir.
- **Finalización mediante manejador:** Al interrumpirse el `pause()`, se salta automáticamente a `manejar_alarma()`, que llama a `PARKING_fin(1)` y al volver a la propia función, se ejecuta un `exit(0)` que termina el proceso, que es recogido por el padre después.
- **Importancia:** Garantiza que la práctica no sea infinita y que el cierre sea «suave», permitiendo que la acera se vacíe antes de destruir los recursos IPC.

### 1. 6. 1. PSEUDOCODIGO FUNCIÓN PROCESO\_AVISADOR

```
{
    // Ignora la señal SIGNINT, para que durante la próxima ejecución no intervenga.
    Signal(SIGINT, SIG_IGN)

    // Llama a la biblioteca indicando una finalización "normal" (valor 1).
    // Esto hace que la biblioteca deje de generar coches nuevos, pero
    // permite a los chóferes terminar de desaparcar los que ya están dentro.
    PARKING_fin(1)

    // Configura el manejador para capturar la señal SIGALARM.
    SignalAction(SIGALARM)

    // Programa el temporizador del sistema 30 segundos que es lo que dura una
    ejecución sin fallos.
    alarm(30)

    // Espera sin consumición de CPU
    pause()

    // Finaliza su ejecución. El proceso principal (padre) lo detectará
    // y sabrá que ha comenzado la fase final de la simulación.
    Exit()
}
```